

Test Automation Guidelines

1. Prerequisites
2. Repository Access
 - 2.1 Prerequisite
 - 2.2 Steps to get access
3. Running Tests Locally
4. LT VDI Setup (Tools & Git)
 - 4.1 Tools on LT VDI
 - 4.2 Git / Azure DevOps (SSH)
5. LT Test Automation Guidelines (General)
6. Automated Test Case Design
7. Robocop Documentation
 - 7.1 Installation
 - 7.2 Basic Rule Checkers
8. Test Cases
9. Testing Anti-Patterns
10. Common Naming Conventions
 - 10.1 Filenames
 - 10.2 Keywords
 - 10.3 Variables
11. Documentation
12. Folder Structure
13. Branching and Merging
 - 13.1 Branching
 - 13.2 Before merging to master/main
 - 13.3 Resolving review comments
 - 13.4 Merging

1. Prerequisites

Software

- Python must be installed (Python **3.13.7**).
- Node.js must be installed for Browser Library:

[📖 Browser library installation and updating instructions](#)

Required libraries

You can refer to the following links to install all required libraries:

- How to get started with test automation development – List of software:

[📖 How to get started with test automation development | List of software](#)

- Browser Library setup:

[📖 Browser library installation and updating instructions](#)

2. Repository Access

2.1 Prerequisite

- You need an **LT email** to get access to the test repository.

2.2 Steps to get access

1. Request license change

- Create a Jira ticket using:

 [Azure-tilausprosessi | Form](#)

- Use the existing ticket as reference:

 [AZURE-2962: TOIVO - TOIVO - Azure DevOps repositories license change](#) CLOSED

2. Request edit rights to test automation repo

- Once the license ticket is resolved, create a ticket on the **LT TA team board**:

 [Test Automation Solution Center | Form](#)

- Use the existing ticket as reference: [https://lahitapiola.atlassian.net/issues?jql=reporter%3DcurrentUser\(\)&selectedIssue=TASC-1706](https://lahitapiola.atlassian.net/issues?jql=reporter%3DcurrentUser()&selectedIssue=TASC-1706)

- Repository:

 https://lt-devtool.visualstudio.com/Test_automation_center/_git/P022-guidewire Connect your Azure DevOps account

- Use service request type: **“Azure access rights”**.
- In the description, specify:
 - All people needing access
 - Which repo
 - Which access level is needed
- Choose component: **service request**.

3. Running Tests Locally

1. Make sure you have all required Python libraries installed:

- In the repo root folder

 https://lt-devtool.visualstudio.com/Test_automation_center/_git/P022-guidewire Connect your Azure DevOps account

run:

```
1 | pip install -r requirements.txt # This has to be followed in CGI VDI/Laptop
```

2. On LT VDI:

- First install/upgrade pip:

```
1 | py -m ensurepip --upgrade
```

- Then install packages:

```
1 | py -m pip install -r requirements.txt
```

3. Running against a new environment

- If you need to run tests against a new environment, make sure you have urls added for that env in `LT_variables.py` file
- Example run command:

```
1 | robot --variablefile resources/variables/LT_variables.py:FUN14 -i test_tag_name  
"Tests\WC\ExposureValidation.robot"
```

4. LT VDI Setup (Tools & Git)

4.1 Tools on LT VDI

- Follow this page for installing tools on LT VDI:

[📖 Guidewire Cloud developer & tester tools](#)

4.2 Git / Azure DevOps (SSH)

To use git commands with SSH:

1. Once you have Git Bash, generate SSH key pair:

```
1 | ssh-keygen -t rsa -b 4096 -C "lahitapiola-email-id"
```

2. The command will show where keys are generated.

3. Add the **public key** to Azure DevOps:

- In the top right corner, click the user/settings icon.
- Click **SSH Keys**.
- Click **Add New Key**.
- Paste the public key from your VDI.

4. Now you can clone using the **SSH URL** from Azure DevOps(`git clone LT-devtool@vs-ssh.visualstudio.com:v3/LT-devtool/Test_automation_center/P022-guidewire`)

5. LT Test Automation Guidelines (General)

We must adhere to LT Test Automation Guidelines:

- [📖 Test Automation Strategy 2025-2026](#)

Repository

- Guidewire Test Automation Framework Repository:

https://lt-devtool.visualstudio.com/Test_automation_center/_git/P022-guidewire?version=G

[Bmain](#) [Connect your Azure DevOps account](#)

Pipeline

- Pipeline link:

https://lt-devtool.visualstudio.com/Test_automation_center/_build?definitionId=6461&a=summary

6. Automated Test Case Design

Automated test cases must follow these rules:

- Each test case **must be independent**.
- Test data must be prepared **automatically**.
- Test cases are **short** with a **clear single goal**.
- Test cases must pass **Robocop static checks**.
- Test cases must be **code reviewed**.
- **Test automation best practices** must be followed.
- For UI testing, follow **Page Object Model (POM)** guidelines.

7. Robocop Documentation

Robocop  is a tool that performs static code analysis of [GitHub - robotframework/robotframework](https://github.com/robotframework/robotframework): Generic automation framework for acceptance testing and RPA code.

It uses the official <https://robot-framework.readthedocs.io/en/stable/> to parse files and runs multiple checks to detect potential errors or violations of code quality standards (linting issues).

7.1 Installation

```
1 | pip install -U robotframework-robocop
```

7.2 Basic Rule Checkers

Robocop rules are grouped into **checkers**. Each checker can scan for multiple related issues (e.g. `LengthChecker` checks for min/max keyword length).

You can refer to a rule by its **name** or **ID** (e.g. `too-long-keyword` or `0501`).

Checker categories and IDs:

- 01: Base
- 02: Documentation
- 03: Naming
- 04: Errors
- 05: Lengths
- 06: Tags
- 07: Comments
- 08: Duplications
- 09: Misc
- 10: Spacing
- 11–50: Reserved for future internal checkers
- 51–99: Reserved for external checkers

Checker types:

- `VisitorChecker` – uses Robot Framework parsing API and Python `ast` module.
- `ProjectChecker` – extends `VisitorChecker` and has a `scan_project` method (called after visiting all files).
- `RawFileChecker` – reads Robot file as text and scans line by line.

Each rule has a unique **4-digit ID**:

- First 2 digits: category ID
- Last 2 digits: rule number

Both **ID** and **name** can be used in include/exclude statements and configurations. Rule severity and parameters can be configured.

8. Test Cases

- No dependencies between test cases (each test case can be run **separately**).
- Test cases should be **easy to read** and understand.
- Use **descriptive**, not too long names.
- **Title Casing** is used often.
- One test should test **one thing**:
 - Small: single component functionality.
 - Large: end-to-end business process.

- Use a **consistent abstraction level** throughout a test.
 - Tests should **not be too long**:
 - ~10 lines is enough in most cases.
 - If the test is too long, increase abstraction (move details into keywords).
-

9. Testing Anti-Patterns

To improve maintainability and performance:

- **No hard-coded values**
Use variables so changes are done in one place.
 - **No sleeps or slowing down entire execution** (e.g. Selenium speed)
Use **dynamic waits** instead.
 - **No absolute paths**
Use **relative paths** so tests work on any machine.
 - **No copy-paste (Don't Repeat Yourself)**
Reuse and extend keywords, use resource files and test templates.
 - **Use configuration files instead of scripts**
For environment-specific runs, use variable/argument files instead of scripts.
 - **Avoid complex logic in user keywords**
If a keyword needs complex if/else or loops, implement it as a **Python library keyword** instead.
 - Do not assume tests are run on a **specific OS**.
-

10. Common Naming Conventions

10.1 Filenames

Test cases

- Use descriptive names where each word starts with an **Upper Case** letter.
- File extension: `.robot`.
- Test cases must be **independent**.
- Test cases should be **self-explanatory** for anyone familiar with the system.
- Test cases should not require modification when **implementation changes**, only when **requirements** change.
- Keywords must take care of implementation details.

- No low-level keywords at test case level
- Test case should have:
 - Clear steps
 - A verification/check
- Test cases should not be too long (not too many keywords).

Test suites

- Use `_` (underscore), **no spaces** in filenames.
- Follow defined **length** guidelines for suites.
- Test suites should not be too long (not too many test cases).
- Create suites for **sets of similar test cases**.
- Avoid massive files:
 - If a suite contains more than ~10 test cases, consider splitting into smaller suites.
 - Similarly, break large high-level suites into nested suites.

Resource files

- Contain **common keywords**, usually more modular and general than suite-specific ones.
- Visible to all test suites that import the resource.
- File extension: `.resource`.

General

- Use **short file names** where possible.

10.2 Keywords

- Capitalize each word.
- Use descriptive names.

User keywords

- In user keyword names, each word should start with an **Upper Case** letter.
- User keywords can have different abstraction levels and scopes.

Scopes:

- **Test suite:**

Keywords used only in a specific suite, often high-level.

Defined in the `Keywords` table of a suite file and visible only in that suite.

- **Resource:**

Common keywords, varying levels of abstraction.

Defined in a resource file and visible to importing suites.

Library keywords

- Complex logic should be implemented as **library keywords** in Python (e.g. loops, complex conditions).
- Python keyword libraries should follow **PEP 8 – Style Guide for Python Code**:
[📄 PEP 8 – Style Guide for Python Code | peps.python.org](https://peps.python.org/pep-0008/)
- Use code inspection tools in your editor (e.g. in PyCharm: `Code > Inspect Code`).

10.3 Variables

- Robot provides built-in variables:
[📖 Robot Framework User Guide](#)
 - Be **consistent** in naming.
 - Variable names can contain **spaces or underscores** – Robot treats them the same.
 - Robot is **case-insensitive**, so `${EXAMPLE VARIABLE}` and `${example_variable}` are the same variable. Avoid mixing styles for the same variable.
 - Recommended convention:
 - All **lowercase** for **local** variables.
 - All **UPPERCASE** for **global** and **suite** variables.
-

11. Documentation

- Each **test suite** must have documentation in the **Settings** section.
- Each **resource** must have documentation in the **Settings** section.

Test files

- Can have suite-level documentation describing:
 - Overall purpose of the suite.
 - Execution details.
 - Link to process in **Confluence**.
 - Links to **Epics and Stories in Jira**.

Test cases

- May need additional documentation, but each test should be written so it is self-explanatory.

User keywords

- Can have documentation explaining usage.

- Document:
 - Return values.
 - Arguments that are not self-explanatory.
- Simple keywords with clear names usually don't need extra docs.

Library files and keywords (Python)

- Documented using **Python docstrings**.
- Documentation should include:
 - How to import the library.
 - For each keyword: explanation of arguments and return values.
 - Examples of usage.
- You can use **libdoc** to generate HTML documentation:

 [Robot Framework User Guide](#)

Keep all documentation up to date.

12. Folder Structure

- Follow LT folder structure guidelines:

 [Robot Framework Style Guide](#)

13. Branching and Merging

- Each test automation engineer uses **their own branch** for test case development.

13.1 Branching

- Always branch **from** `main`.
- Always **pull latest code** from Azure DevOps before creating a new branch.
- Use **Jira Ticket ID + short description** in the branch name.

13.2 Before merging to master/main

Code must be **reviewed**. Before requesting review:

- Pass **Robocop** checks.
- Pass:

```
1 | robocop --dryrun
```

- Make sure **correct labels** are in place.
- Ensure documentation is **up to date**:

- Links to Jira and Confluence are added.
- Create a **pull request** in Azure DevOps.
- Assign a **reviewer**.

13.3 Resolving review comments

- Address all comments, update the code and, if needed, update documentation.

13.4 Merging

- After merging:
 - Delete the branch in **Azure DevOps**.
 - Delete the **local branch**.
 - Do **not** push new code to a branch that has already been merged.